

Indexing

▼ Intro

This chapter introduces **indexing structures** for files, which are **auxiliary access structures** that speed up record retrieval without altering the physical storage of the primary data file. Any field in a file can be used to create an index, and multiple indexes—on single or multiple fields—can exist for the same file.

- **Purpose of indexes:**

- Provide secondary access paths.
- Enable efficient search based on the indexed fields.
- Lead to pointers to disk blocks containing the desired records.

▼ Single-Level Ordered Indexing

What does “Single-Level Ordered” mean?

- **Single-Level:**

- The index has only **one layer** or level of pointers.
- Each entry in the index points directly to the disk block(s) where the actual records are stored.
- This is in contrast to **multilevel indexes** (like B+-trees), where the index points to another index, creating multiple layers.

- **Ordered:**

- The entries in the index are **stored in sorted order** based on the indexing field.
- Because the index is sorted, you can use **binary search** to quickly locate the entry for a given value, rather than scanning the entire index linearly.

So, a **single-level ordered index** is just a **sorted list of index entries** that directly points to the records in the data file. It's simple and efficient for files that aren't too large.

▼ Types Of Single-Level Ordered Indexing

▼ Primary Indexes

Visual Example

Data File (3,000 blocks):

text

Copy

Download

```

Block 1: [Record 1, Record 2, ..., Record 10]
Block 2: [Record 11, Record 12, ..., Record 20]
...
Block 3000: [Record 29991, Record 29992, ..., Record 30000]

```

Index File (45 blocks):

text

Copy

Download

```

Index Block 1: [Entry for Data Block 1, Entry for Data Block 2, ..., Entry for Data Block 68]
Index Block 2: [Entry for Data Block 69, Entry for Data Block 70, ..., Entry for Data Block 136]
...
Index Block 45: [Entry for Data Block 2953, ..., Entry for Data Block 3000]

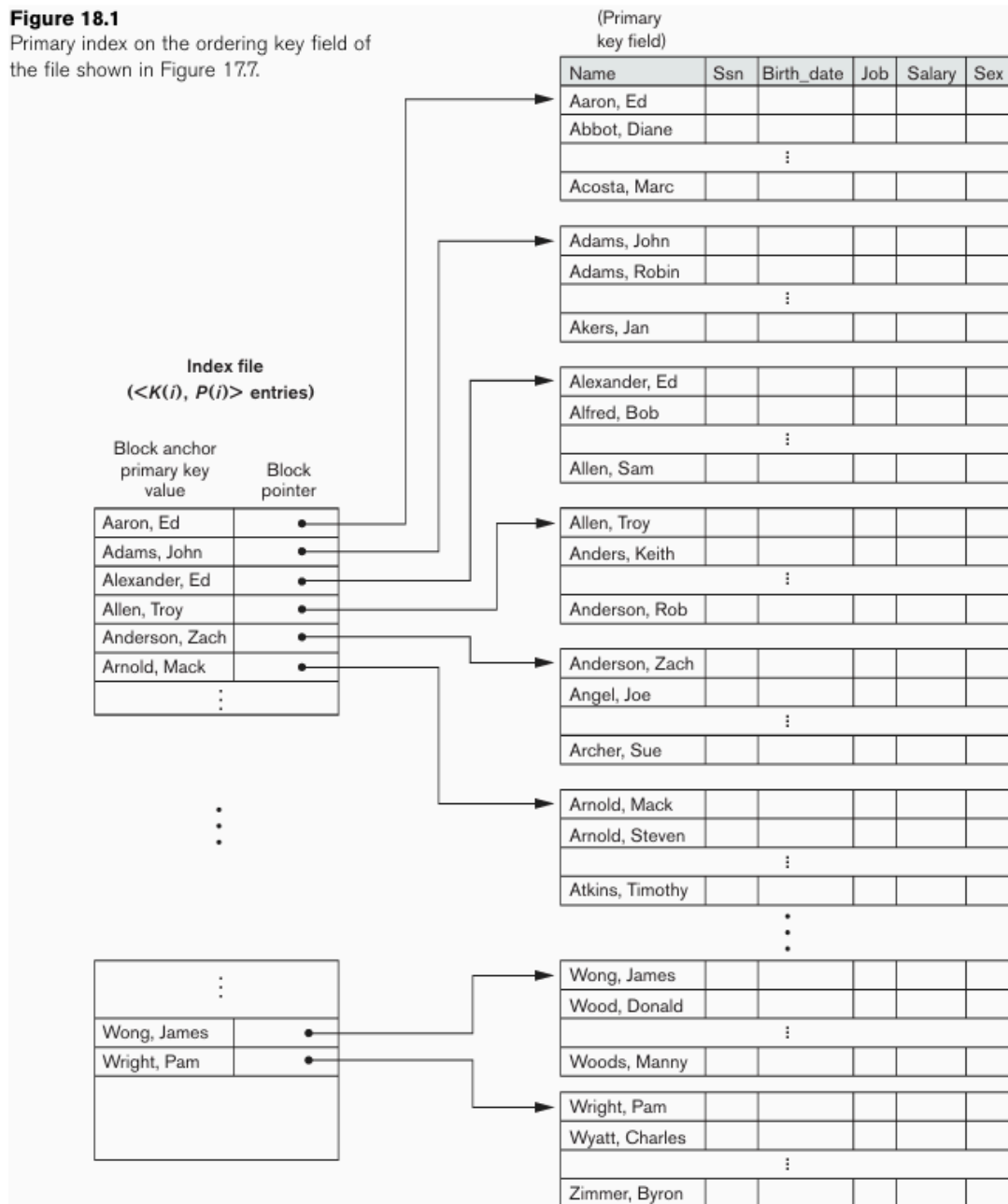
```

Example 1. Suppose that we have an ordered file with $r = 30,000$ records stored on a disk with block size $B = 1024$ bytes. File records are of fixed size and are unspanned, with record length $R = 100$ bytes. The blocking factor for the file would be $bfr = \lfloor (B/R) \rfloor = \lfloor (1024/100) \rfloor = 10$ records per block. The number of blocks needed for the file is $b = \lceil (r/bfr) \rceil = \lceil (30000/10) \rceil = 3000$ blocks. A binary search on the data file would need approximately $\lceil \log_2 b \rceil = \lceil (\log_2 3000) \rceil = 12$ block accesses.

Now suppose that the ordering key field of the file is $V = 9$ bytes long, a block pointer is $P = 6$ bytes long, and we have constructed a primary index for the file. The size of each index entry is $R_i = (9 + 6) = 15$ bytes, so the blocking factor for the index is $bfr_i = \lfloor (B/R_i) \rfloor = \lfloor (1024/15) \rfloor = 68$ entries per block. The total number of index entries r_i is equal to the number of blocks in the data file, which is 3000. The number of index blocks is hence $b_i = \lceil (r_i/bfr_i) \rceil = \lceil (3000/68) \rceil = 45$ blocks. To perform a binary search on the index file would need $\lceil (\log_2 b_i) \rceil = \lceil (\log_2 45) \rceil = 6$ block accesses. To search for a record using the index, we need one additional block access to the data file for a total of $6 + 1 = 7$ block accesses—an improvement over binary search on the data file, which required 12 disk block accesses.

Figure 18.1

Primary index on the ordering key field of the file shown in Figure 17.7.



Primary Index: Overview

A **primary index** is an **ordered access structure** built on the **ordering key (primary key)** of an **ordered data file**. Its purpose is to **efficiently locate and access records** in a file without scanning the entire file.

- **Structure of a Primary Index:**

- The index is itself an **ordered file** with **two fields per entry**:
 1. **Key value** ($K(i)$) – the primary key of the **first record in a block** (also called the **anchor record**).
 2. **Pointer** ($P(i)$) – the address of the corresponding **disk block** in the data file.
 - **One entry per data block** in the file.
 - Example entry: $\langle K(i) = (Aaron, Ed), P(i) = \text{address of block 1} \rangle$
 - **Anchor Record:**
 - The **first record in each block** of the data file.
 - The primary index stores the key of this record as the index key.
-

Dense vs Sparse Indexes

- **Dense Index:**
 - Contains an index entry for **every record** in the data file.
 - **Sparse Index (Primary Index):**
 - Contains an entry for **each disk block**, not each record.
 - Fewer entries, smaller index size → faster searches.
-

Advantages of a Primary Index

1. **Smaller Index File:**
 - Fewer entries than records in the data file.
 - Each entry has only **two fields**, so more entries fit in a single block.
2. **Efficient Search:**
 - **Binary search** on the smaller index file → fewer block accesses than binary search on the full data file.
 - Once the correct block is found via the index, only **one additional access** is needed to retrieve the record.

Example:

- Data file: 30,000 records, 10 records per block → 3,000 blocks.
 - Binary search on **data file**: ~12 block accesses ($\log_2(3000)$).
 - Primary index entries: 3,000 entries, 68 entries per block → 45 index blocks.
 - Binary search on **index file**: $\log_2(45) \approx 6$ block accesses + 1 access to data block = **7 block accesses** → almost **half** the original.
-

How to Search Using a Primary Index

1. Perform **binary search** on the index to find entry i such that: $K(i) \leq K < K(i+1)$
 $K(i) \leq K < K(i+1)$
 - K = key value to search.
 2. Retrieve the **data block** at address $P(i)$.
 3. Search within the block (records may be sequentially scanned).
-

Problems / Limitations

1. Insertion:

- Inserting a record in its correct order requires **shifting records** and updating **anchor records in the index**.
- Can be partially solved with:
 - **Unordered overflow file** (insert into overflow instead of main file).
 - **Linked list of overflow records** per block (records within block + overflow can be sorted).

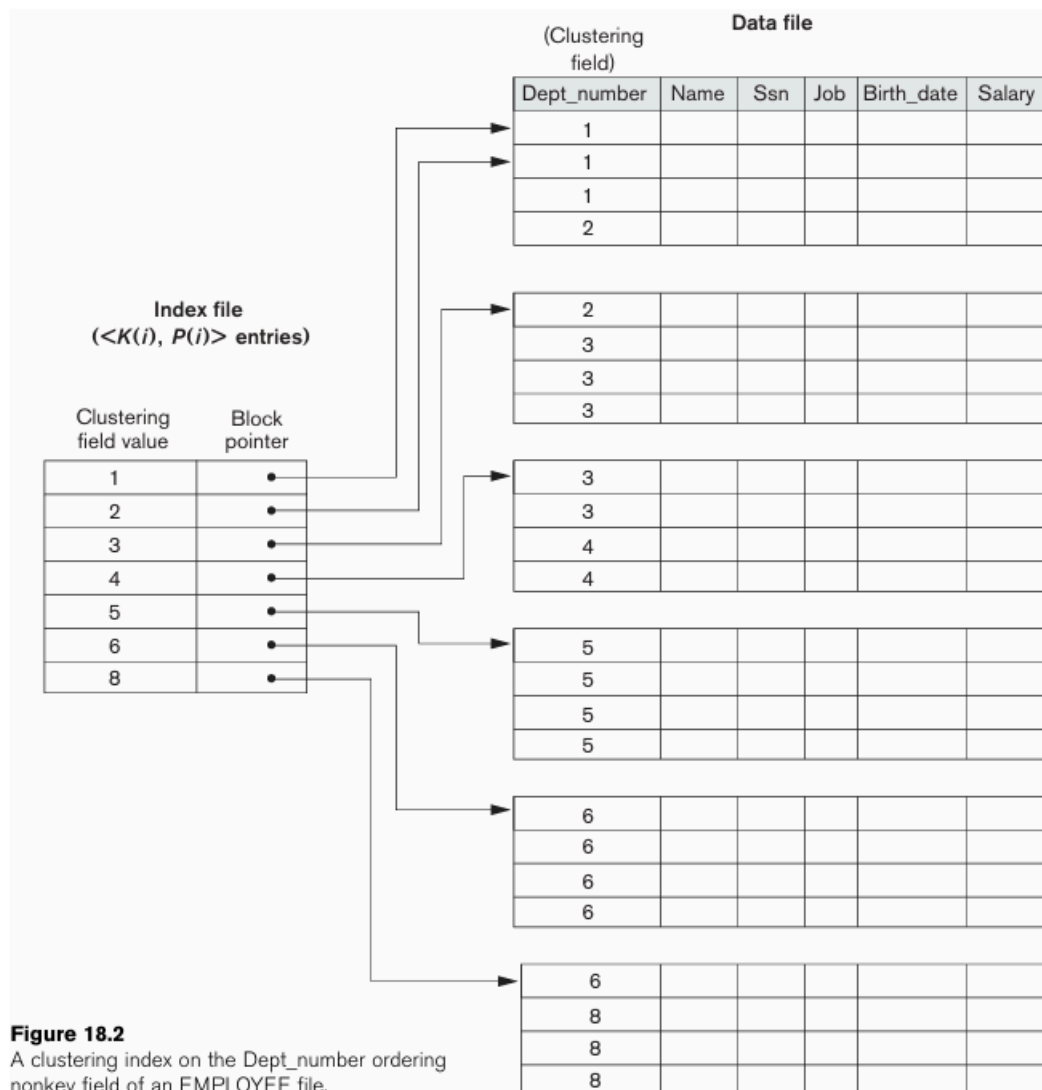
2. Deletion:

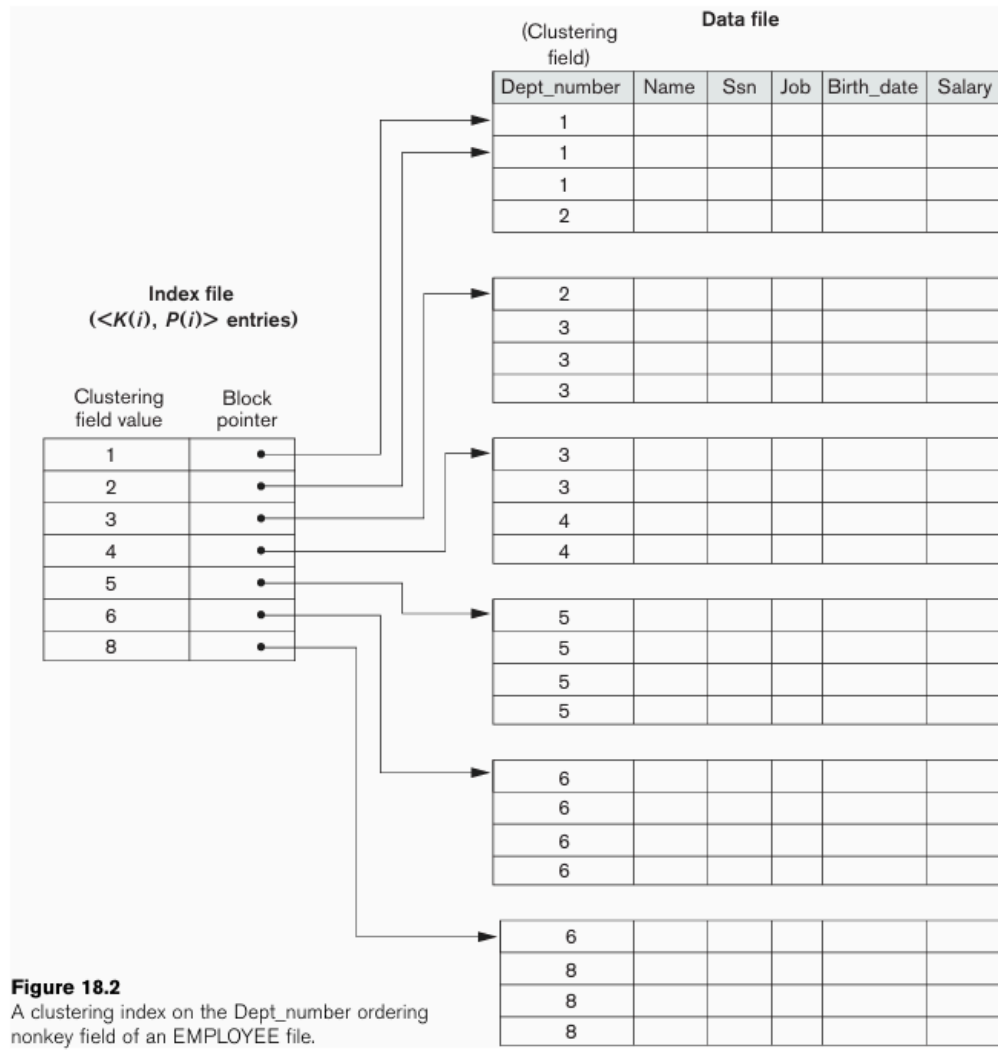
- Requires updating anchor records or using **deletion markers** in the block.
-

✓ Key Points to Remember

- Primary index is **sparse** (one entry per block).
- Each entry = `<first record's key, pointer to block>`.
- Significantly **reduces block accesses** for searches.
- Maintains **physical ordering** of the file.
- Insertions and deletions are more complicated due to ordering.

▼ Clustering Indexes





A **clustering index** is used when a file is **physically ordered on a nonkey field**, called the **clustering field**.

- **Clustering Field:**

- A field that **may have duplicate values** across records.
- Example: `department_id` in an employee file—many employees can belong to the same department.

- **Clustered File:**

- A data file physically **ordered by the clustering field**.

Structure of a Clustering Index

- Like a primary index, a clustering index is an **ordered file with two fields**:
 1. **Clustering field value** – the distinct value of the field.
 2. **Pointer to data block** – points to the **first block containing records with that value**.
- **One entry per distinct value** of the clustering field.
- Because the field is **non-unique**, multiple records can share the same value, unlike a primary index.
- This makes a clustering index **sparse (nondense)**.

Example entry:

```
Dept 10 → Block 1 (all employees of Dept 10 start here)
Dept 20 → Block 3
Dept 30 → Block 5
```

Advantages

- Efficient retrieval of **all records sharing the same value** of the clustering field.
- Particularly useful for **range queries** or queries that return multiple records for the same value.

Problems and Solutions

- **Insertion/Deletion Issues:**
 - Since records are physically ordered, adding a new record may require shifting records.
- **Common Solutions:**
 1. Reserve a **whole block or cluster of contiguous blocks** for each distinct value.

2. Place all records with the same clustering value in these blocks, making insertion and deletion simpler.

Comparison with Primary Index

Feature	Primary Index	Clustering Index
Ordering Field	Unique (primary key)	Non-unique (clustering field)
Entry per	Data block	Distinct value of clustering field
Density	Sparse	Sparse
Retrieval	Single record	Multiple records sharing same value
Physical File Ordering	By primary key	By clustering field

Search Mechanism

- Similar to primary index: search the **index** to find the **pointer** to the block containing the first record of the desired value.
- Unlike hashing, the search uses **actual field values**, not hash codes.

▼ Secondary Indexes

Note : Primary index is not shown here.

Figure 18.4

A dense secondary index (with block pointers) on a nonordering key field of a file.

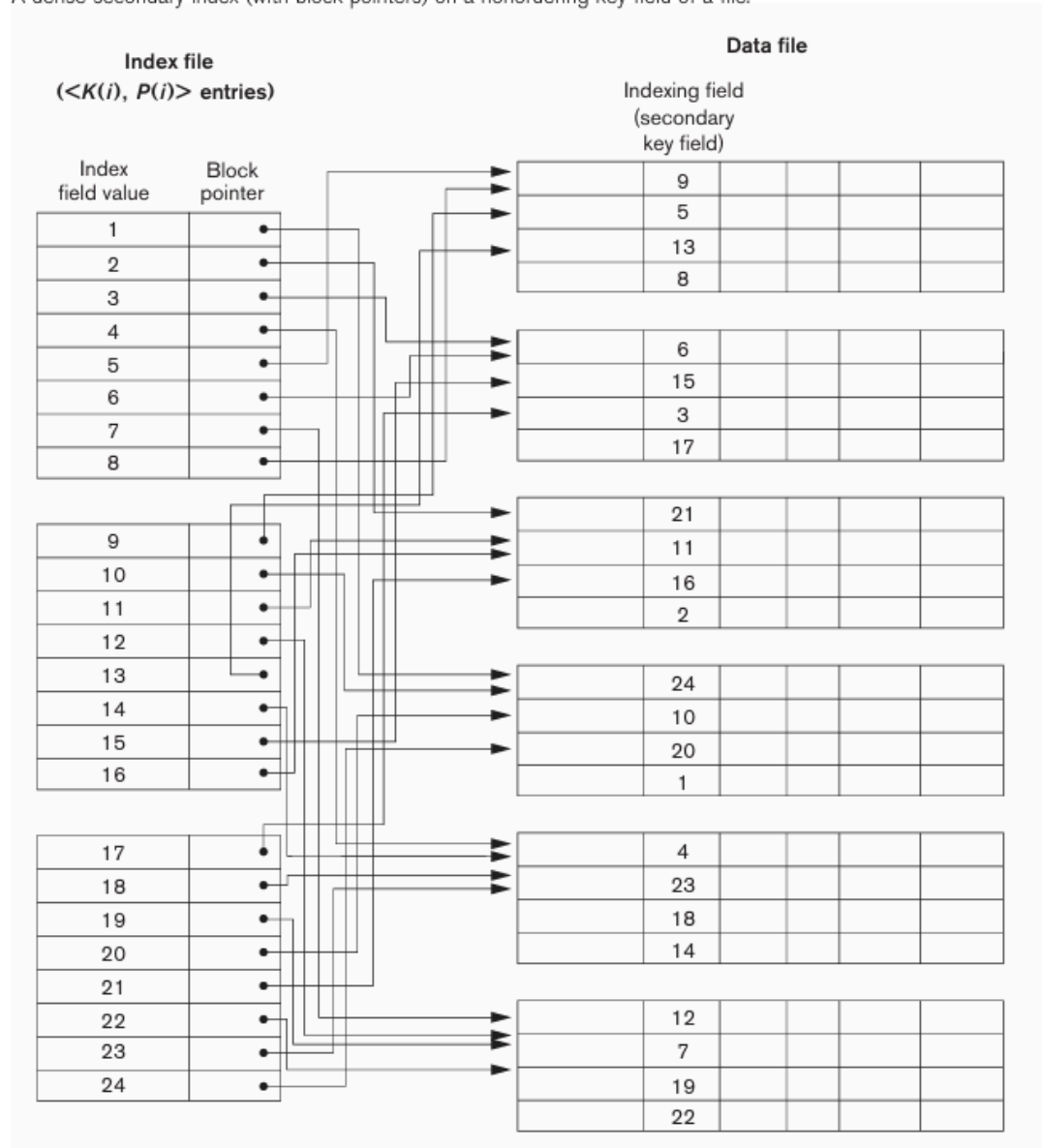
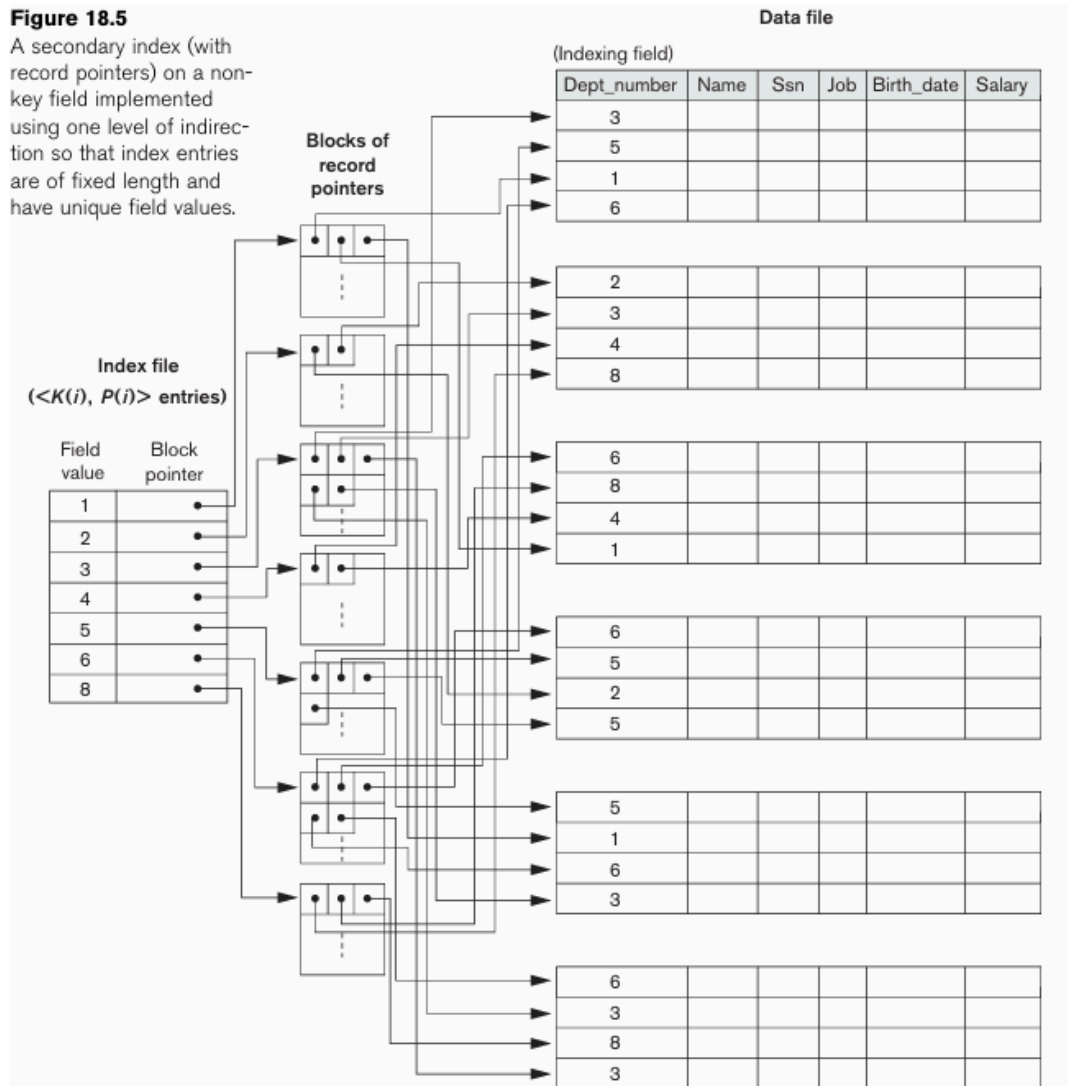


Figure 18.5

A secondary index (with record pointers) on a non-key field implemented using one level of indirection so that index entries are of fixed length and have unique field values.



A secondary index is an **additional, alternative way** to access records in a data file. Think of it like this:

- The **primary key** is the main official identifier (like a Student ID).
- A **secondary index** is like an index in the back of a book for another important term (like "Email Address" or "Last Name"). You can have many of them.

Key Characteristics:

- Can be created on **any field** in the table, not just the primary key.
- The data file does **NOT** need to be physically ordered by this field.

- You can create **many different secondary indexes** on the same file for different fields.
-

Types of Secondary Indexes

The text describes two main types, based on the field they are built on.

1. Secondary Index on a UNIQUE Key Field

- **Field Type:** A field with a unique value for every record (e.g., Email, Social Security Number). This is called a **secondary key**.
- **How it Works:** It is a **Dense Index**. This means:
 - There is **one index entry for every single record** in the data file.
 - Each entry contains: `<Key_Value, Pointer_to_Record>`
 - The index entries are sorted by the key value.

Visual Example:

- **Data File** (unordered by email):
 - Block 1: [Record A (ali@...), Record B (maria@...)]
 - Block 2: [Record C (zoe@...), Record D (ben@...)]
- **Secondary Index on Email** (sorted):
 - [ali@... → Ptr to Record A]
 - [ben@... → Ptr to Record D]
 - [maria@... → Ptr to Record B]
 - [zoe@... → Ptr to Record C]

2. Secondary Index on a NON-UNIQUE (Duplicate) Field

- **Field Type:** A field where many records can have the same value (e.g., Department, City, Job Title).
- **How it Works:** There are three common ways to handle the duplicates:

Option 1: Dense Duplicate Entries

- Create multiple index entries for the same key value.
- *Example:* [Sales -> Ptr to Rec 5], [Sales -> Ptr to Rec 9], [Sales -> Ptr to Rec 21]
- **Problem:** The index can become very large.

Option 2: Variable-Length Entries

- A single index entry holds a list of all pointers for that key.
- *Example:* [Sales -> <Ptr5, Ptr9, Ptr21>]
- **Problem:** Hard to manage because entries have different sizes.

Option 3: Extra Level of Indirection (Most Common)

- A single, fixed-length index entry points to a **bucket** (a separate disk block).
- The **bucket** contains a list of all the pointers to the actual records.
- *Example:*
 - **Index:** [Sales -> Ptr to Bucket 77]
 - **Bucket 77:** <Ptr to Rec 5, Ptr to Rec 9, Ptr to Rec 21>
- **This is the preferred method** because it's efficient and keeps the main index simple and fixed-length.

Performance Comparison (The Examples)

The text uses numbers to show why indexes are so powerful.

Search Method	Data File	Primary Index	Secondary Index (on unique field)
Description	Search the 3,000 data blocks directly.	Search a small index of data block anchors.	Search an index of every record.
Index Size	N/A	45 blocks	442 blocks
Block Accesses	Linear Search: ~1500	7 (6 for index + 1 for data)	10 (9 for index + 1 for data)

Key Takeaways:

- **Secondary Index vs. No Index:** A secondary index (10 accesses) is a **massive improvement** over a linear scan (1500 accesses).
- **Secondary Index vs. Primary Index:** A secondary index (10 accesses) is slightly slower than a primary index (7 accesses). **Why?** Because a secondary index is **dense** (it has an entry for every one of the 30,000 records), making it larger. A primary index is **non-dense** (it only has an entry for each of the 3,000 blocks), making it smaller and faster to search.

Summary & Key Differences from Primary Index

Feature	Primary Index	Secondary Index
Data File Order	Must be ordered by the index field.	Can be unordered by the index field.
Index Type	Non-Dense (Sparse). One entry per block .	Dense . One entry per record .
Number Allowed	Only one per file.	Many per file (on different fields).
Pointer Points To	A whole block of records.	A single record (or a bucket of pointers).
Size	Smaller.	Larger.
Speed	Faster to search.	Slightly slower to search, but still very fast.

Table 18.2 Properties of Index Types

Type of Index	Number of (First-level) Index Entries	Dense or Nondense (Sparse)	Block Anchoring on the Data File
Primary	Number of blocks in data file	Nondense	Yes
Clustering	Number of distinct index field values	Nondense	Yes/no ^a
Secondary (key)	Number of records in data file	Dense	No
Secondary (nonkey)	Number of records ^b or number of distinct index field values ^c	Dense or Nondense	No

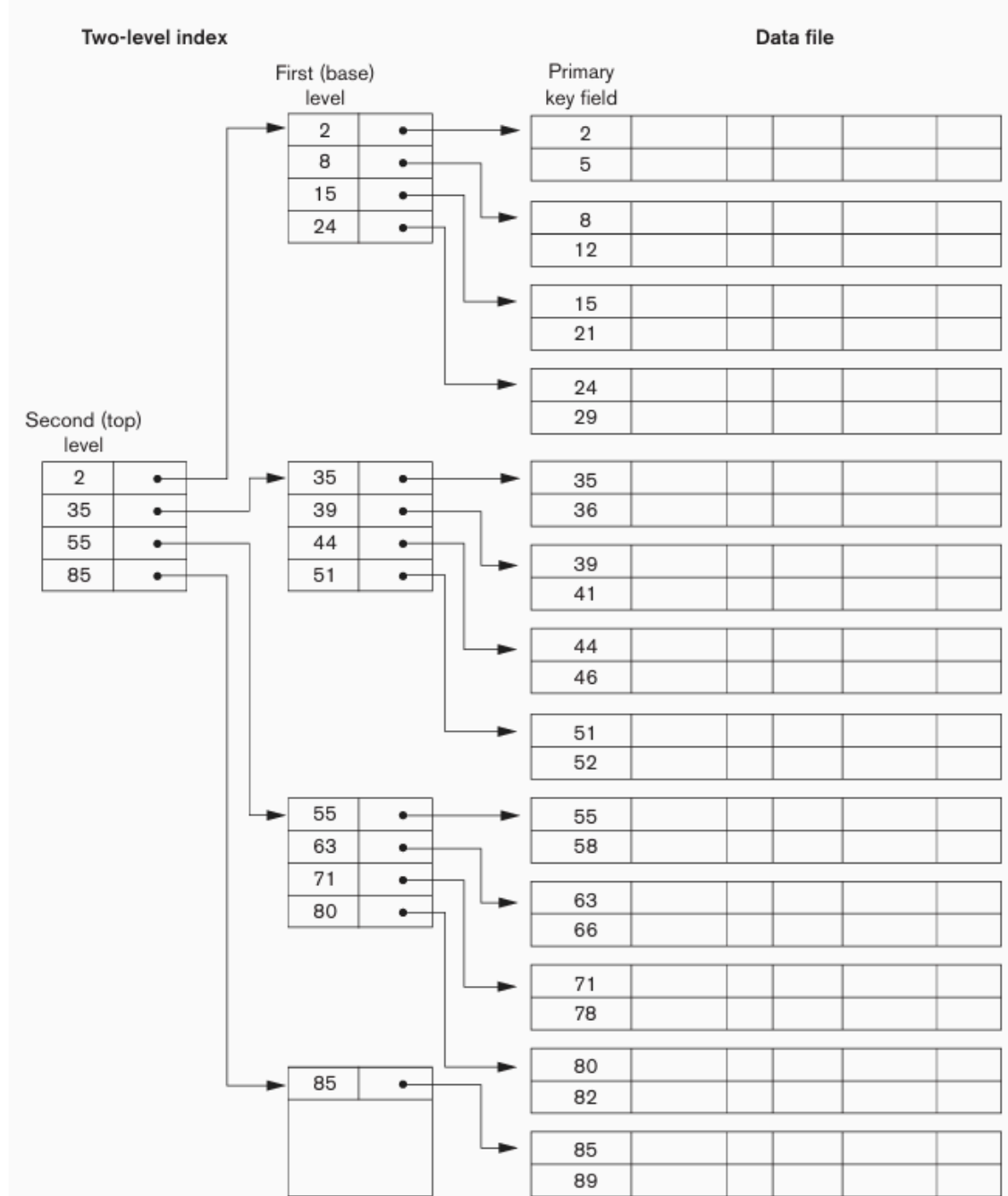
^aYes if every distinct value of the ordering field starts a new block; no otherwise.

^bFor option 1.

^cFor options 2 and 3.

▼ Multilevel Indexes

A two-level primary index resembling ISAM (Indexed Sequential Access Method) organization.



Example 3. Suppose that the dense secondary index of Example 2 is converted into a multilevel index. We calculated the index blocking factor $bfr_i = 68$ index entries per block, which is also the fan-out fo for the multilevel index; the number of first-level blocks $b_1 = 442$ blocks was also calculated. The number of second-level blocks will be $b_2 = \lceil (b_1/fo) \rceil = \lceil (442/68) \rceil = 7$ blocks, and the number of third-level blocks will be $b_3 = \lceil (b_2/fo) \rceil = \lceil (7/68) \rceil = 1$ block. Hence, the third level is the top level of the index, and $t = 3$. To access a record by searching the multilevel index, we must access one block at each level plus one block from the data file, so we need $t + 1 = 3 + 1 = 4$ block accesses. Compare this to Example 2, where 10 block accesses were needed when a single-level index and binary search were used.

Why Multilevel Indexes?

The problem with a single-level index is that you have to use a **binary search** to find your entry. A binary search is good, but we can do much better.

- **Binary Search:** Cuts the search space in half each time $\rightarrow \log_2(b_1)$ block accesses.
- **Multilevel Index:** Cuts the search space by a much larger factor (the *fan-out*) each time $\rightarrow \log_{fo}(b_1)$ block accesses.

The goal is to reduce the number of disk reads needed to find a record.

How a Multilevel Index is Built

It works by building a **hierarchy of indexes on top of each other**, like a pyramid.

Level 1 (Base Level): This is your original, single-level index (e.g., the dense secondary index from the previous example with 442 blocks).

Level 2: You create a **primary index for the Level 1 index file**.

- Since Level 1 is a sorted file, you can build a non-dense (sparse) index for it.
- This new index (Level 2) will have only one entry **per block** of Level 1.
- If Level 1 has 442 blocks, Level 2 will have 442 entries.

Level 3: You create a primary index for the **Level 2** index file.

- This continues until you reach a level that fits entirely in a **single block**. This is the **top level**.

The Power of Fan-out (fo)

The **fan-out** (fo) is the blocking factor for the index (bfr_i). It's the number of index entries that fit in one block. This is the key to the speed.

- In our examples, $fo = 68$.
- This means that at each step in the search, you eliminate 67/68ths of the remaining search space, instead of just half!

The Search Process

To find a record using a multilevel index with t levels:

1. Read the **top-level** block (Level t). It points to the correct block in the level below.
2. Read **one block** from Level $t-1$.
3. Continue down, reading **one block per level**.
4. Finally, read the **one data block** containing your record.

Total Block Accesses = t (for the index) + 1 (for the data) = $t + 1$

Performance Comparison (The Example)

Let's use the dense secondary index from before:

Metric	Single-Level (Binary Search)	Multilevel
Index Size	442 blocks	3 levels
Search Cost	$\log_2(442) \approx 9$ index accesses + 1 data = 10 total	3 index accesses + 1 data = 4 total

This is a massive improvement, cutting the search cost by more than half.

Key Concepts & Terminology

- **Fan-out (f_o)**: The number of index entries per block. This determines how quickly the search narrows down.
- **Top Level**: The highest level of the index that fits in a single block. The search always starts here.
- **Indexed Sequential File (ISAM)**: A classic file organization that uses an ordered data file with a multilevel primary index. IBM's ISAM is a famous implementation using cylinder and track indexes.

Summary: Multilevel vs. Single-Level

Feature	Single-Level Index	Multilevel Index
Search Method	Binary Search	Hierarchical Descent
Search Speed	$\log_2(b_i)$	$\log_{f_o}(b_i)$ (Much faster)
Structure	One flat file	A tree-like hierarchy of files
Typical Access	9-12 block accesses	3-4 block accesses

▼ Dynamic Multilevel Index

A tree-based index that can reorganize itself automatically to stay balanced as data changes.

That's exactly what **B-Trees** and **B+ Trees** do.

Why We Need Trees for Indexing

In a large database:

- Even the *index file* can become huge.
- A single-level or fixed multilevel index becomes **inefficient** when new records are added or deleted — it may need complete reorganization.

👉 Solution: Use a **tree-based index** that automatically balances itself.

That's where **B-Trees** and **B+ Trees** come in.

▼ Search Trees

What Is a Search Tree?

A **search tree** is a **data structure** that helps you find records efficiently, based on some **search key** (e.g., Employee_ID, Roll_No, etc.).

It is conceptually similar to a multilevel index — but with more flexibility.

Rules (always maintained)

1. Keys inside each node are **sorted**:

$K_1 < K_2 < K_3 < \dots < K_{q-1}$

2. The **pointers divide the key space**:

- For all values X in subtree pointed by P_i :

- If $i = 1$: $X < K_1$
- If $1 < i < q$: $K_{i-1} < X < K_i$
- If $i = q$: $X > K_{q-1}$

Figure 18.8

A node in a search tree with pointers to subtrees below it.

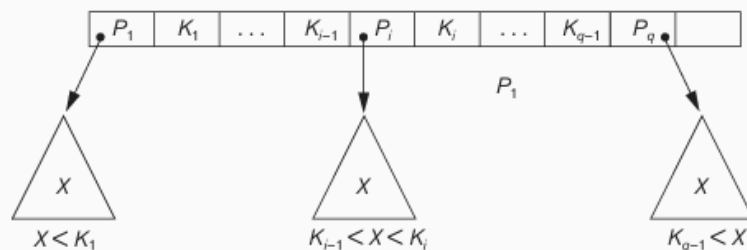
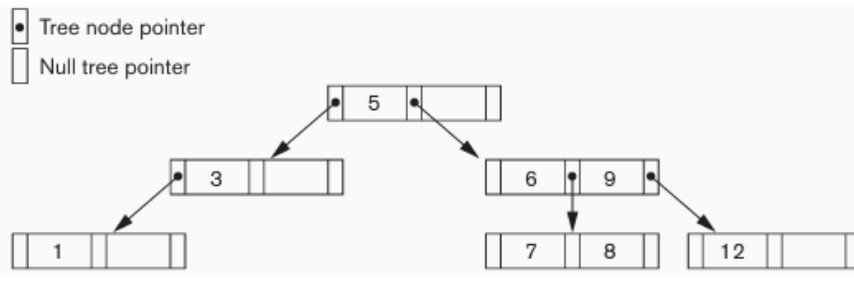


Figure 18.9

A search tree of order $p = 3$.



Problem with Basic Search Trees

They are **not automatically balanced**.

After many insertions/deletions:

- Some paths can get deeper than others (unbalanced tree).
- Searching might take **longer for some records**.
- Some nodes might become **half-empty** (wasting space).

That's why we need **balanced search trees**, i.e., **B-Trees**.

▼ B - trees

What Is a B-Tree?

A **B-Tree** (short for *Balanced Tree*) is a **balanced version of a search tree** used for **dynamic multilevel indexing**.

It automatically maintains:

- ✓ Balance (all leaves at the same level)
- ✓ Efficiency (each node at least half full)
- ✓ Fast updates (insertions/deletions don't require full reorganization)

Each **internal node** looks like:

```
<P1, <K1, Pr1>, P2, <K2, Pr2>, ..., <Kq-1, Prq-1>, Pq>
```

Where:

- P_i → pointer to a child node
- K_i → key value
- P_{ri} → pointer to the actual record (or data block)
- $q \leq p$

◆ Rules (Properties)

1. Keys inside node are **sorted** ($K_1 < K_2 < \dots < K_{q-1}$).
2. **Search Rule:**

For subtree pointed to by P_i :

- $K_{i-1} < X < K_i$ for middle pointers
- $X < K_1$ for P_1
- $X > K_{q-1}$ for P_q

- Each node has **at most p pointers**.
- Each **non-root, non-leaf** node has **at least $\lceil p/2 \rceil$ pointers**.
- Root has at least 2 pointers (unless it's the only node).
- A node with q pointers has $q-1$ keys.
- All leaf nodes are at the same level** (balanced).

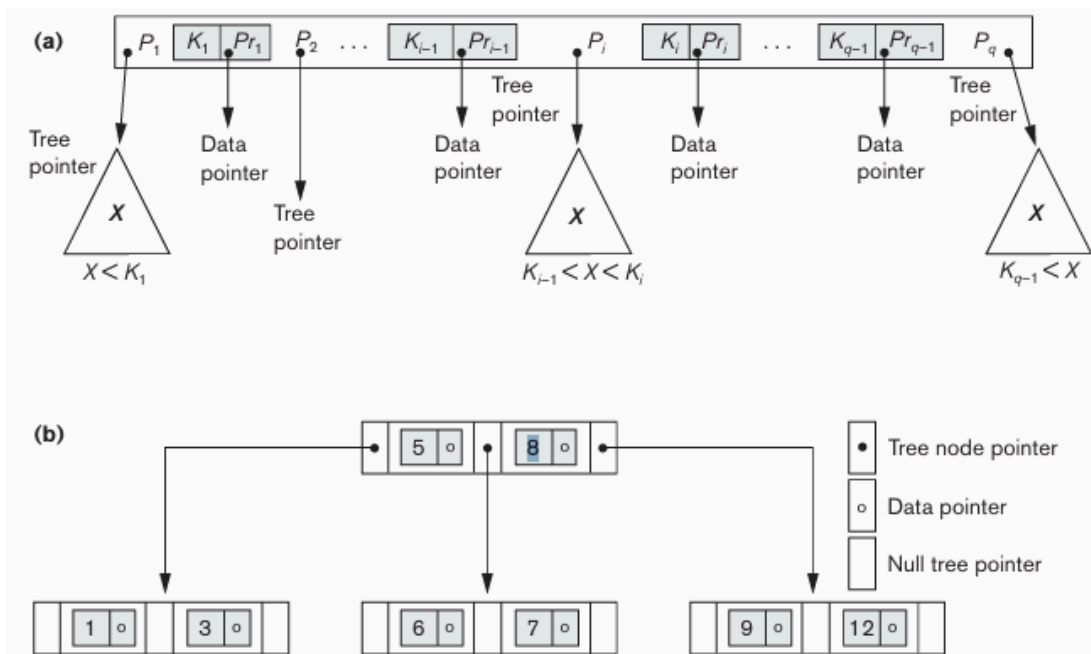


Figure 18.10

B-tree structures. (a) A node in a B-tree with $q-1$ search values. (b) A B-tree of order $p=3$. The values were inserted in the order 8, 5, 1, 7, 3, 12, 9, 6.

Example 4. Suppose that the search field is a nonordering key field, and we construct a B-tree on this field with $p = 23$. Assume that each node of the B-tree is 69 percent full. Each node, on the average, will have $p * 0.69 = 23 * 0.69$ or approximately 16 pointers and, hence, 15 search key field values. The **average fan-out** $fo = 16$. We can start at the root and see how many values and pointers can exist, on the average, at each subsequent level:

Root:	1 node	15 key entries	16 pointers
Level 1:	16 nodes	240 key entries	256 pointers
Level 2:	256 nodes	3840 key entries	4096 pointers
Level 3:	4096 nodes	61,440 key entries	

At each level, we calculated the number of key entries by multiplying the total number of pointers at the previous level by 15, the average number of entries in each node. Hence, for the given block size, pointer size, and search key field size, a two-level B-tree holds $3840 + 240 + 15 = 4095$ entries on the average; a three-level B-tree holds 65,535 entries on the average.

B-trees are sometimes used as **primary file organizations**. In this case, *whole records* are stored within the B-tree nodes rather than just the $\langle \text{search key}, \text{record pointer} \rangle$ entries. This works well for files with a relatively *small number of records* and a *small*

8. Transition to B+ Trees

B-Trees are powerful but have a limitation:

Data pointers are stored **in both internal and leaf nodes**, making range queries slower.

👉 That's why most modern DBMSs use **B+ Trees**, which improve upon this by:

- Keeping **data pointers only in leaves**.
- Linking leaves together for **faster range searches**.

▼ B+ - Trees

A B+-Tree has **two types of nodes**:

◆ (A) Internal Nodes (index levels)

Used for **navigating** the tree — they contain **search key values** and **tree pointers** (to child nodes).

They *do not* contain actual data.

Structure:

$\langle P_1, K_1, P_2, K_2, \dots, P_{q-1}, K_{q-1}, P_q \rangle$ where $q \leq p$

- P_i = pointer to a child node
- K_i = key (search value)

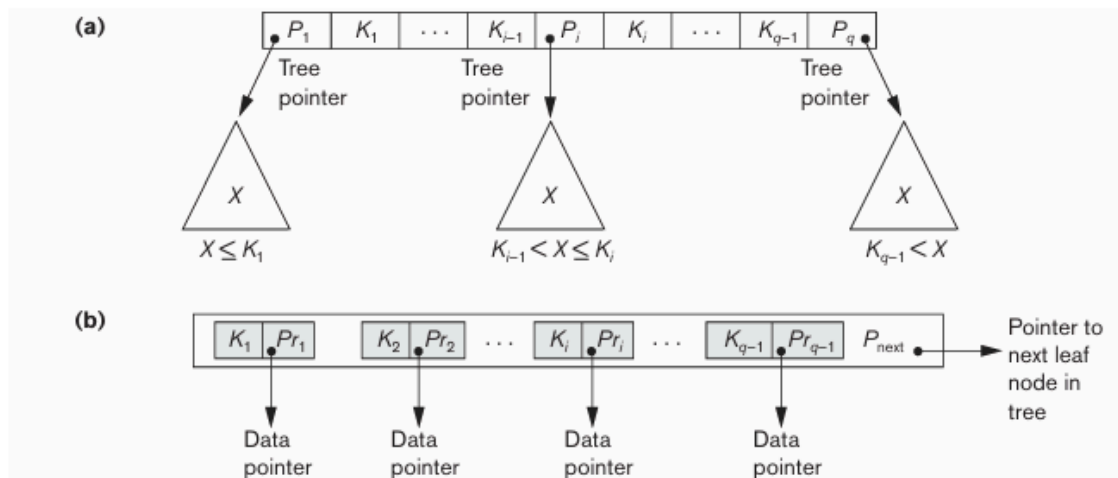
Properties:

1. Keys are sorted: $K_1 < K_2 < \dots < K_{q-1}$
2. Search rule:
 - $K_{i-1} < X \leq K_i \rightarrow$ go to subtree P_i
3. Max p pointers per node (order = p)
4. Each non-root node must have at least $\lceil p/2 \rceil$ pointers
5. Root has at least 2 pointers (if not a leaf)
6. If node has q pointers $\rightarrow$ it has $q-1$ keys

✓ So internal nodes **guide** the search but **don't hold data**.

Figure 18.11

The nodes of a B⁺-tree. (a) Internal node of a B⁺-tree with $q - 1$ search values.
 (b) Leaf node of a B⁺-tree with $q - 1$ search values and $q - 1$ data pointers.



◆ The Formula

$$(p * P) + ((p - 1) * V) \leq B$$

◆ Meaning of Each Symbol

Symbol	Meaning	Explanation	
p	Number of pointers (or index entries) that can fit in one block	It tells how many index records fit in one index block.	
P	Size of a pointer (in bytes)	The size of each pointer that points to either another index block or a data block.	
V	Size of the key field (in bytes)	The size of the actual key value stored in the index entry.	
B	Block size (in bytes)	The total size of one disk block (or page).	

◆ What the Formula Represents

Each index record contains:

- One **pointer** (to the next level or to a data block)
- One **key value**

But — note that the **first pointer** in an index block doesn't have a preceding key (it points to values smaller than the first key).

So, if a block contains **p** pointers, it will contain **(p - 1)** key values.

Hence, the **total space required** in a block is:

$$(p * P) + ((p - 1) * V)$$

And this total must fit **within one disk block (B)**:

$$(p * P) + ((p - 1) * V) \leq B$$

◆ 1 Background: Multilevel Index

A multilevel index has two kinds of index blocks:

1. **Upper-level index blocks** — contain key + pointer pairs, pointing to the next level of index blocks.
2. **Leaf-level index blocks** — contain key + record pointer pairs, pointing to **actual data records** in the data file.

So, the structure at the **leaf level** is slightly different from upper levels — that's why we get a **different formula** for leaf blocks.

◆ 2 Meaning of Each Symbol

Symbol	Meaning	Explanation
p_{leaf}	Number of key–pointer pairs that fit in a leaf block	(like how many records the leaf index can hold)
P_r	Size of the record pointer (bytes)	Each key in the leaf level points to an actual record in the data file
V	Size of the key value (bytes)	Each record key is stored with its pointer
P	Size of the block pointer to the next leaf block	Leaf blocks are linked (for sequential access), so we store one extra pointer at the end
B	Block size (bytes)	Total size of one disk page

◆ 3 The Formula

At the leaf level, each entry takes up $(P_r + V)$ bytes.

If a leaf block can store p_{leaf} entries, their total size is:

$$p_{\text{leaf}} * (P_r + V)$$

But we also need one extra pointer (P) to link this leaf block to the next one (for sequential access).

So, total space used is:

$$(p_{\text{leaf}} * (P_r + V)) + P$$

This must fit within the block size (B):

$$(p_{\text{leaf}} * (P_r + V)) + P \leq B$$

◆ 4 Purpose

This formula is used to compute how many (key, record pointer) pairs can be stored in one leaf-level index block.

This helps determine:

- how many leaf blocks are needed for all records
- how many levels the index needs overall.

Example 5. To calculate the order p of a B^+ -tree, suppose that the search key field is $V = 9$ bytes long, the block size is $B = 512$ bytes, a record pointer is $Pr = 7$ bytes, and a block pointer is $P = 6$ bytes. An internal node of the B^+ -tree can have up to p tree pointers and $p - 1$ search field values; these must fit into a single block. Hence, we have:

$$\begin{aligned}(p * P) + ((p - 1) * V) &\leq B \\(P * 6) + ((P - 1) * 9) &\leq 512 \\(15 * p) &\leq 521\end{aligned}$$

We can choose p to be the largest value satisfying the above inequality, which gives $p = 34$. This is larger than the value of 23 for the B-tree (it is left to the reader to compute the order of the B-tree assuming same size pointers), resulting in a larger fan-out and more entries in each internal node of a B^+ -tree than in the corresponding B-tree. The leaf nodes of the B^+ -tree will have the same number of values and pointers, except that the pointers are data pointers and a next pointer. Hence, the order p_{leaf} for the leaf nodes can be calculated as follows:

$$\begin{aligned}(p_{\text{leaf}} * (Pr + V)) + P &\leq B \\(p_{\text{leaf}} * (7 + 9)) + 6 &\leq 512 \\(16 * p_{\text{leaf}}) &\leq 506\end{aligned}$$

It follows that each leaf node can hold up to $p_{\text{leaf}} = 31$ key value/data pointer combinations, assuming that the data pointers are record pointers.

As with the B-tree, we may need additional information—to implement the insertion and deletion algorithms—in each node. This information can include the type of node (internal or leaf), the number of current entries q in the node, and pointers to the parent and sibling nodes. Hence, before we do the above calculations for p and p_{leaf} , we should reduce the block size by the amount of space needed for all such information. The next example illustrates how we can calculate the number of entries in a B^+ -tree.

Example 6. Suppose that we construct a B⁺-tree on the field in Example 5. To calculate the approximate number of entries in the B⁺-tree, we assume that each node is 69 percent full. On the average, each internal node will have $34 * 0.69$ or approximately 23 pointers, and hence 22 values. Each leaf node, on the average, will hold $0.69 * p_{\text{leaf}} = 0.69 * 31$ or approximately 21 data record pointers. A B⁺-tree will have the following average number of entries at each level:

Root:	1 node	22 key entries	23 pointers
Level 1:	23 nodes	506 key entries	529 pointers
Level 2:	529 nodes	11,638 key entries	12,167 pointers
Leaf level:	12,167 nodes	255,507 data record pointers	

For the block size, pointer size, and search field size given above, a three-level B⁺-tree holds up to 255,507 record pointers, with the average 69 percent occupancy of nodes. Compare this to the 65,535 entries for the corresponding B-tree in Example 4. This is the main reason that B⁺-trees are preferred to B-trees as indexes to database files.

▼ Indexes on Multiple Keys

What Problem Are We Solving?

So far, we've discussed indexes that are built on **one attribute** (e.g., `Ssn`, `Dno`, `Age`).

But in real queries, we often search using **multiple conditions** together, like:

|"List all employees in department number 4 whose age is 59."

Here, the condition involves **two nonkey attributes**: `Dno` and `Age`.

Neither one alone uniquely identifies a record.

◆ 2 The Problem with Separate Single-Attribute Indexes

You could have:

- an **index on Dno**
- an **index on Age**

Then, for this query:

- From the **Dno** index, you get all employees in department 4.
- From the **Age** index, you get all employees aged 59.
- You **intersect** these two sets.

✓ This gives the correct result — but ✗ it can be **inefficient**.

If there are many employees in department 4, and many aged 59, you may fetch **large sets** before narrowing to just a few combined matches.

◆ 3 The Better Solution: Composite Index

If queries often use **combinations** of fields like **(Dno, Age)** together, we can create an index on **both fields as one combined key** — e.g.:

Index on (Dno, Age) \text{Index on } \langle \text{Dno, Age} \rangle

Index on (Dno, Age)

This is a **composite index** (or **multikey index**).

Each index entry will look like:

(Dno, Age)	Pointer
(1, 23)	→ record pointer
(1, 45)	→ record pointer
(2, 30)	→ record pointer
(4, 59)	→ record pointer

◆ 4 How the DBMS Uses It

If a composite index is built on **(Dno, Age)** :

- A query with **both fields** (`Dno=4` and `Age=59`) can be satisfied **directly** from the index.
- A query with **only Dno** (`Dno=4`) can also benefit — because the index is ordered first by `Dno` .
- But a query with **only Age** (`Age=59`) **cannot** use this index efficiently (since the first sorting key is `Dno`).

So the **order of attributes** in the composite index matters a lot.

◆ 5 Composite Key ≠ Multiple Indexes

Don't confuse these:

Concept	Description
Multiple single-field indexes	Separate index on each field (<code>Dno</code> and <code>Age</code> individually)
Composite (multikey) index	One index that combines both fields (<code>Dno, Age</code>) into one ordered key

Composite indexes handle **multi-attribute conditions more efficiently**, especially when those attributes are often queried together.

▼ Ordered Index on Multiple Attributes

1 Ordered Index on Multiple Attributes (Composite Key Index)

🧩 Concept:

If we build an index on a **combination of fields**, e.g.,

`<Dno, Age>`

`<Dno, Age>`

then each **search key** becomes a **tuple** like `<4, 59>` .

These tuples are **ordered lexicographically** (just like dictionary words).

🧠 Lexicographic order means:

- Compare the **first attribute** (`Dno`) first.
- If it's the same, compare the **next attribute** (`Age`).

So:

```
<3, anything> comes before <4, anything>
<4, 18> < <4, 19> < <4, 20> ...
```

It's the same as how words "CAT" < "DOG" < "FROG" are ordered.

✓ Advantage:

- Works like any normal **B+-tree** index.
- Supports **range queries** (e.g., `Dno=4 and Age>50`).
- Easy to extend to many attributes, e.g. `<A1, A2, A3>` .

⚠ Limitation:

- Only efficient **if the leftmost attributes** are used in the query.
 - Index on `<Dno, Age>` helps with `Dno=4` or `(Dno=4 AND Age=59)` .
 - But **not** with queries on `Age` alone.

◆ 2 Partitioned Hashing

🧩 Concept:

This is an extension of **hash indexing** for **composite keys**.

Instead of one hash value, we generate **multiple hash parts**, one for each attribute, and then **concatenate** them.

Example:

- Key = `<Dno, Age>`
- Dno → 3-bit hash
- Age → 5-bit hash

- Combined 8-bit address → `Dno_hash || Age_hash`

If:

- `Dno=4 → 100`
- `Age=59 → 10101`

then:

```
Bucket address = 10010101
```

So, record with `<Dno=4, Age=59>` is stored in bucket `10010101`.

✓ Advantages:

- Can easily extend to multiple attributes.
- Can retrieve combinations like `<Dno=4, Age=59>` quickly.
- No need for separate indexes on each attribute.

⚠ Drawback:

- **No range queries** possible (only equality conditions).
 - You can search for exact matches like `Dno=4 AND Age=59`.
 - But not for `Age > 40`.

◆ 3 Grid Files

🧩 Concept:

A **grid file** is like a **multi-dimensional hash structure** — great for multi-key access.

Each attribute has its own **linear scale** (axis).

Example with two attributes:

Axis	Represents
X-axis	Dno (Department No.)
Y-axis	Age

You then divide each scale into intervals that distribute records **uniformly**.

Each cell in the grid corresponds to a **bucket** (a storage block containing records).

So:

- (Dno=4, Age=59) might map to **cell (1,5)** → points to bucket **B₁₅**.

✓ Advantages:

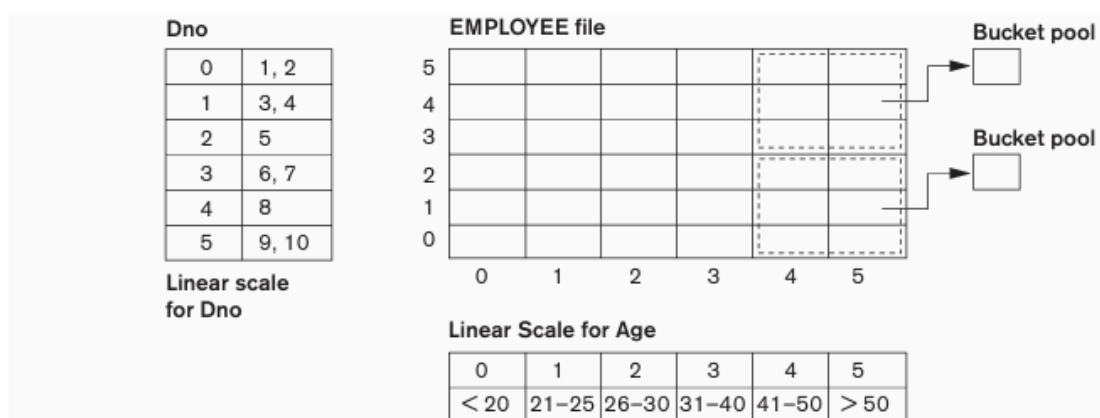
- Excellent for **range queries** across multiple attributes.
 - e.g., **Dno ≤ 5 AND Age > 40** → just access the set of grid cells that match this range.
- Works for **n attributes** → becomes an n-dimensional grid.

⚠ Drawbacks:

- **Space overhead** due to grid array.
- **Reorganization cost** when data changes dynamically.

Figure 18.14

Example of a grid array on Dno and Age attributes.



▼ Other Types of Indexes

◆ 1 Hash Indexes

🧩 Concept:

- A **hash index** is an index built on a search key using **hashing**.
- The index is **secondary**, meaning the main file can be stored sequentially or in any other order.
- Each index entry is of the form:

```
<K, Pr> or <K, P>
```

- **K** = key value
- **Pr** = pointer to the record
- **P** = pointer to the block containing the record

🔄 How it works:

1. Apply a **hash function** on the key **K** to get a **bucket number**.
2. Access the bucket to find **<K, Pr>**.
3. Use **Pr** to go to the actual record in the data file.

Example:

- EMPLOYEE file ordered by Name.
- Hash on **Emp_id** using: sum of digits modulo 10.
- **Emp_id = 51024 → hash = 2 → go to bucket 2**.

✅ Advantages:

- Fast access for equality queries: **WHERE Emp_id = 51024**

- Scales well for large tables

⚠ Limitations:

- No direct support for **range queries** (`Emp_id > 50000`).

◆ 2 Bitmap Indexes

🧩 Concept:

- A **bitmap index** uses **bits (0/1)** to represent presence of a value for a column.
- Efficient for columns with **few distinct values** (low cardinality) and **large tables**.

How it works:

1. Number rows from 0 to n.
2. For a column `c` and a value `v`, create an **n-bit vector**:
 - Bit = 1 if the row has `c=v`, else 0.
3. If a column has m distinct values, create **m bitmaps**.

Example (Sex column in EMPLOYEE):

Row_id	Sex
0	M
1	F
2	M
3	F
4	F
5	M
6	M
7	F

- Bitmap for `Sex = M`: `1 0 1 0 0 1 1 0`

- Bitmap for `Sex = F`: `0 1 0 1 1 0 0 1`

✓ Querying:

- **Single column:** `Sex = F` → return row_ids where bitmap = 1
- **Multiple columns:** use **AND/OR** on bitmaps:

```
Sex = F AND Zipcode = 30022
→ intersect bitmaps
→ get row_ids 1 and 3
```

- **Negation:** use **NOT** (flip bits)

✓ Advantages:

- Very **space efficient**: 1 million rows → 1 million bits = 125 KB per bitmap
- Efficient **AND/OR/NOT operations** on multiple keys

⚠ Limitations:

- Expensive updates: inserting/deleting rows requires updating bitmaps.
- Best for **low-cardinality columns** (e.g., Sex, Yes/No flags).

💡 Special Use:

- **B+-trees leaf nodes:** For nonkey fields that occur in many records, a bitmap can replace a list of record pointers, saving space.

♦ Summary:

Index Type	Best For	Query Support	Storage / Notes	
Hash Index	Exact match on key	Equality only	Secondary structure, fast lookup	
Bitmap Index	Low-cardinality columns	Equality, AND/OR/NOT queries	Very compact, efficient for large tables	

Index Type	Best For	Query Support	Storage / Notes	
Bitmap in B+	Frequent nonkey values	Equality	Saves space compared to record pointers	

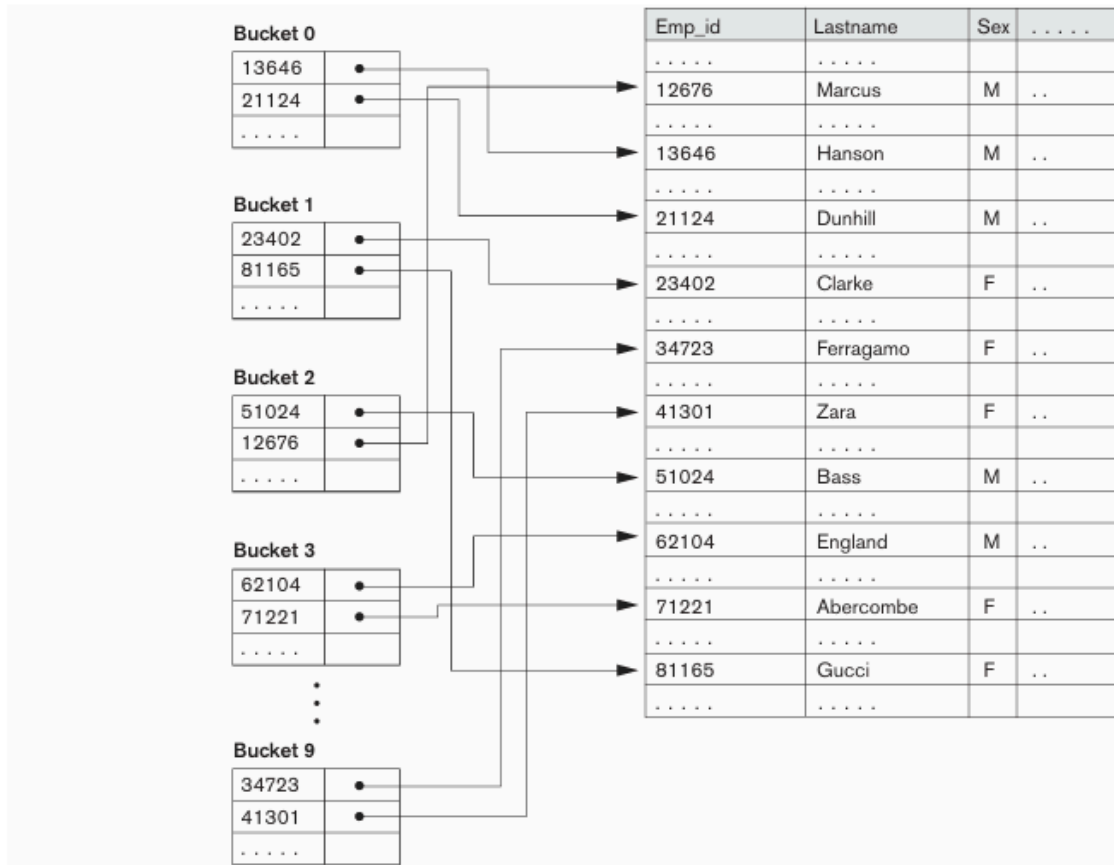


Figure 18.15
Hash-based indexing.



Figure 18.16
Bitmap indexes for
Sex and Zipcode

▼ Bitmaps for B+-Tree Leaf Nodes

◆ Normal B+-tree Leaf Node for Nonkey Field

- Suppose you have a B+-tree on a **nonkey field** like `City`.
- Each leaf node stores entries like:

```
<Ki, [Pr1, Pr2, ..., Prn]>
```

- `Ki` = value of the field (e.g., "New York")
- `[Pr1, Pr2, ..., Prn]` = pointers to **all records** that have `City = Ki`.

⚠ Problem:

If a value occurs in **many records** (e.g., thousands of people live in "New York"), the list of pointers `[Pr1, Pr2, ..., Prn]` can become very large. This wastes space in the leaf node.

◆ Using a Bitmap Instead

- Instead of storing **thousands of record pointers**, we store a **bitmap (bitvector)**.

How it works:

1. Suppose the table has **n rows**.

2. Create a bitvector of length **n**.
3. For each row:
 - If the row has `City = "New York"`, set the corresponding bit to 1.
 - Otherwise, set it to 0.

Example:

Row	City
0	New York
1	Boston
2	New York
3	Boston

- Bitvector for `"New York"` = `1 0 1 0`
- Bitvector for `"Boston"` = `0 1 0 1`

✓ Benefit:

- Instead of storing 2 pointers for `"New York"` (Pr0, Pr2), we store a **bitmap of 4 bits**.
- If there are **millions of rows**, storing a bitmap is still **much smaller** than storing millions of pointers.

◆ Summary

Method	Space Needed (for n rows)
Record pointers	4 bytes × number of matching rows
Bitmap	1 bit × n rows

- If a value occurs **very frequently**, the bitmap is **smaller and more efficient**.
- This is why in B+-trees on nonkey fields, **bitmaps can replace large lists of record pointers**.

▼ Function-Based Index

What Is Function-Based Indexing?

A **function-based index** is an index **not on the column itself**, but on the **result of a function applied to the column(s)**.

Normally, a B+-tree or other index works on **raw column values**. If you apply a function in a query (e.g., `UPPER(Lname)`), a normal index cannot be used, and the database may do a **full table scan**.

A **function-based index** solves this by storing the **function result in the index**.

◆ Examples

1. Uppercase Last Name Index

```
CREATE INDEX upper_ix ON Employee (UPPER(Lname));
```

- Function: `UPPER(Lname)` → converts last names to uppercase.
- Query:

```
SELECT First_name, Lname  
FROM Employee  
WHERE UPPER(Lname) = 'SMITH';
```

✓ Without the function-based index: Full table scan.

✓ With the index: The query can directly use the index, even though the query applies `UPPER()`.

1. Arithmetic Expression Index

```
CREATE INDEX income_ix ON Employee(Salary + (Salary*Comm  
ission_pct));
```

- Function: `Salary + (Salary*Commission_pct)` → total income including commission.
- Query:

```
SELECT First_name, Lname
FROM Employee
WHERE ((Salary*Commission_pct) + Salary ) > 15000;
```

- The database can still use the index **even if the query uses the fields in a different order**, as long as the expression matches.

1. Conditional Unique Index

```
CREATE UNIQUE INDEX promo_ix ON Orders
(
  CASE WHEN Promotion_id = 2 THEN Customer_id ELSE NULL
END,
  CASE WHEN Promotion_id = 2 THEN Promotion_id ELSE NULL
END
);
```

- Function: `CASE WHEN Promotion_id = 2 THEN Customer_id ELSE NULL END`
- Effect: Only applies the unique constraint **for Promotion_id = 2**.
- Why it works: Oracle **does not store index entries where all keys are NULL**. So rows where `Promotion_id ≠ 2` are ignored in the index.

This allows enforcing **conditional uniqueness**.

◆ Key Takeaways

- Function-based indexes **enable indexing on expressions** rather than just raw columns.
- Useful for:
 - Queries with functions like `UPPER`, `TRIM`, etc.
 - Calculated columns (arithmetic expressions).
 - Conditional or filtered constraints (e.g., only certain rows enforce uniqueness).
- The database can use this index to **avoid full table scans**.

If you want, I can make a **tiny diagram showing a B+-tree with function-based indexing** so you can visualize it easily.

Do you want me to do that?

You said:

A function-based index solves this by storing the function result in the index. what ??

Okay! Let's clarify that in plain terms.

♦ What It Means

Normally, an **index** stores the values of a column to help you find rows quickly. For example, a B+-tree index on `Lname` stores:

```
SMITH → points to row 7
JONES → points to row 3
LEE   → points to row 12
```

Now, imagine you often search using `UPPER(Lname)` :

```
SELECT * FROM Employee WHERE UPPER(Lname) = 'SMITH';
```

- The database can't use the normal `Lname` index directly, because the index stores `Smith`, `Jones`, `Lee` (original case), **not** the uppercase version.
- Without a special index, it scans every row and applies `UPPER(Lname)` to check — very slow.

◆ How Function-Based Index Helps

A **function-based index** stores the **result of the function** instead of the raw column.

For `UPPER(Lname)`, the index looks like this:

```
SMITH → points to row 7  
JONES → points to row 3  
LEE   → points to row 12
```

Notice: It stores **the uppercase version directly**, not the original text.

So now the query:

```
SELECT * FROM Employee WHERE UPPER(Lname) = 'SMITH';
```

- Can **directly look up 'SMITH'** in the index.
- No need to scan the table and calculate `UPPER()` for every row.
- Much faster!

✓ Summary:

A function-based index is just an index **on the output of a function** applied to a column. It precomputes and stores the function results, so the database can quickly search using that function.